Algoritmos de ordenação

Implementação e testes de mesa

Kauã da Silva Santos
Programação - atividade individual

Usei o vetor **[5, 3, 4, 1, 2]** nos cinco algoritmos. Assim fica mais fácil comparar o trabalho que cada um faz para chegar ao resultado **[1, 2, 3, 4, 5]**.

Como fiz a contagem

Contei apenas comparações entre valores do vetor, incluindo a chave e o pivô. Testes de índices, condições dos laços e chamadas recursivas não entram nessa conta. Os índices começam em zero.

Considerei troca quando dois elementos em posições diferentes são permutados. Trocar uma posição com ela mesma não entra na contagem. No Insertion, contei os deslocamentos para a direita separadamente. No Merge, contei as escritas no vetor original durante a intercalação; as cópias para os vetores auxiliares não entram nesse total.

As tabelas seguem a saída do código entregue. C, T e M são totais acumulados: comparações, trocas e movimentos. No Insertion e no Merge, o vetor pode ter valores repetidos temporariamente, porque a chave ou as cópias auxiliares guardam os valores que ainda serão recolocados.

Resumo dos resultados

Algoritmo	Comparações	Trocas	Movimentos*
Bubble sort	10	8	0
Selection sort	10	4	0
Insertion sort	10	0	8
Merge sort	6	0	12
Quick sort	7	4	0

*Movimentos: deslocamentos no Insertion e escritas no Merge. Os demais algoritmos registram suas permutas na coluna de trocas. Esses movimentos não são uma contagem de todas as atribuições do programa.

Bubble sort

Comparei os vizinhos e troquei os pares fora de ordem. A cada passagem, o maior valor restante foi para o final. Usei a versão sem parada antecipada.

Entrada: [5, 3, 4, 1, 2] | Saída: [1, 2, 3, 4, 5]

Passo	Ação	Vetor	C	T	M
0	Entrada	[5, 3, 4, 1, 2]	0	0	0
1	5 > 3: troca	[3, 5, 4, 1, 2]	1	1	0
2	5 > 4: troca	[3, 4, 5, 1, 2]	2	2	0
3	5 > 1: troca	[3, 4, 1, 5, 2]	3	3	0
4	5 > 2: troca	[3, 4, 1, 2, 5]	4	4	0
5	3 > 4: mantem	[3, 4, 1, 2, 5]	5	4	0
6	4 > 1: troca	[3, 1, 4, 2, 5]	6	5	0
7	4 > 2: troca	[3, 1, 2, 4, 5]	7	6	0
8	3 > 1: troca	[1, 3, 2, 4, 5]	8	7	0
9	3 > 2: troca	[1, 2, 3, 4, 5]	9	8	0
10	1 > 2: mantem	[1, 2, 3, 4, 5]	10	8	0
11	Resultado	[1, 2, 3, 4, 5]	10	8	0

Total: 10 comparações, 8 trocas e 0 movimentos.

Selection sort

Procurei o menor valor da parte ainda não ordenada e o coloquei na próxima posição. As linhas de busca não alteram o vetor; a troca acontece depois de terminar a busca.

Entrada: [5, 3, 4, 1, 2] | Saída: [1, 2, 3, 4, 5]

Passo	Ação	Vetor	C	T	M
0	Entrada	[5, 3, 4, 1, 2]	0	0	0
1	3 < 5: novo minimo	[5, 3, 4, 1, 2]	1	0	0
2	4 < 3: mantem minimo	[5, 3, 4, 1, 2]	2	0	0
3	1 < 3: novo minimo	[5, 3, 4, 1, 2]	3	0	0
4	2 < 1: mantem minimo	[5, 3, 4, 1, 2]	4	0	0
5	Posiciona minimo: i=0, menor=3	[1, 3, 4, 5, 2]	4	1	0
6	4 < 3: mantem minimo	[1, 3, 4, 5, 2]	5	1	0
7	5 < 3: mantem minimo	[1, 3, 4, 5, 2]	6	1	0
8	2 < 3: novo minimo	[1, 3, 4, 5, 2]	7	1	0
9	Posiciona minimo: i=1, menor=4	[1, 2, 4, 5, 3]	7	2	0
10	5 < 4: mantem minimo	[1, 2, 4, 5, 3]	8	2	0
11	3 < 4: novo minimo	[1, 2, 4, 5, 3]	9	2	0
12	Posiciona minimo: i=2, menor=4	[1, 2, 3, 5, 4]	9	3	0
13	4 < 5: novo minimo	[1, 2, 3, 5, 4]	10	3	0
14	Posiciona minimo: i=3, menor=4	[1, 2, 3, 4, 5]	10	4	0
15	Resultado	[1, 2, 3, 4, 5]	10	4	0

Total: 10 comparações, 4 trocas e 0 movimentos.

Insertion sort

Guardei cada chave, desloquei os valores maiores para a direita e inseri a chave no espaço encontrado. Esta versão usa deslocamentos: por isso, o total de trocas é zero.

Entrada: [5, 3, 4, 1, 2] | Saída: [1, 2, 3, 4, 5]

Passo	Ação	Vetor	C	T	M
0	Entrada	[5, 3, 4, 1, 2]	0	0	0
1	Chave=3	[5, 3, 4, 1, 2]	0	0	0
2	5 > chave 3: sim	[5, 3, 4, 1, 2]	1	0	0
3	Desloca para indice 1	[5, 5, 4, 1, 2]	1	0	1
4	Insere chave no indice 0	[3, 5, 4, 1, 2]	1	0	1
5	Chave=4	[3, 5, 4, 1, 2]	1	0	1
6	5 > chave 4: sim	[3, 5, 4, 1, 2]	2	0	1
7	Desloca para indice 2	[3, 5, 5, 1, 2]	2	0	2
8	3 > chave 4: nao	[3, 5, 5, 1, 2]	3	0	2
9	Insere chave no indice 1	[3, 4, 5, 1, 2]	3	0	2
10	Chave=1	[3, 4, 5, 1, 2]	3	0	2
11	5 > chave 1: sim	[3, 4, 5, 1, 2]	4	0	2
12	Desloca para indice 3	[3, 4, 5, 5, 2]	4	0	3
13	4 > chave 1: sim	[3, 4, 5, 5, 2]	5	0	3
14	Desloca para indice 2	[3, 4, 4, 5, 2]	5	0	4
15	3 > chave 1: sim	[3, 4, 4, 5, 2]	6	0	4
16	Desloca para indice 1	[3, 3, 4, 5, 2]	6	0	5
17	Insere chave no indice 0	[1, 3, 4, 5, 2]	6	0	5
18	Chave=2	[1, 3, 4, 5, 2]	6	0	5
19	5 > chave 2: sim	[1, 3, 4, 5, 2]	7	0	5
20	Desloca para indice 4	[1, 3, 4, 5, 5]	7	0	6
21	4 > chave 2: sim	[1, 3, 4, 5, 5]	8	0	6
22	Desloca para indice 3	[1, 3, 4, 4, 5]	8	0	7
23	3 > chave 2: sim	[1, 3, 4, 4, 5]	9	0	7
24	Desloca para indice 2	[1, 3, 3, 4, 5]	9	0	8
25	1 > chave 2: nao	[1, 3, 3, 4, 5]	10	0	8
26	Insere chave no indice 1	[1, 2, 3, 4, 5]	10	0	8
27	Resultado	[1, 2, 3, 4, 5]	10	0	8

Total: 10 comparações, 0 trocas e 8 movimentos.

Merge sort

Dividi o vetor até chegar a partes com um elemento. Depois intercalei as partes em ordem. Esta versão usa vetores auxiliares e cópias, sem permutar diretamente dois elementos.

Entrada: [5, 3, 4, 1, 2] | Saída: [1, 2, 3, 4, 5]

Passo	Ação	Vetor	C	T	M
0	Entrada	[5, 3, 4, 1, 2]	0	0	0
1	Divide [0,4] no meio 2	[5, 3, 4, 1, 2]	0	0	0
2	Divide [0,2] no meio 1	[5, 3, 4, 1, 2]	0	0	0
3	Divide [0,1] no meio 0	[5, 3, 4, 1, 2]	0	0	0
4	Intercala [0,0] e [1,1]	[5, 3, 4, 1, 2]	0	0	0
5	5 <= 3: copia menor	[3, 3, 4, 1, 2]	1	0	1
6	Copia restante esquerdo	[3, 5, 4, 1, 2]	1	0	2
7	Intercala [0,1] e [2,2]	[3, 5, 4, 1, 2]	1	0	2
8	3 <= 4: copia menor	[3, 5, 4, 1, 2]	2	0	3
9	5 <= 4: copia menor	[3, 4, 4, 1, 2]	3	0	4
10	Copia restante esquerdo	[3, 4, 5, 1, 2]	3	0	5
11	Divide [3,4] no meio 3	[3, 4, 5, 1, 2]	3	0	5
12	Intercala [3,3] e [4,4]	[3, 4, 5, 1, 2]	3	0	5
13	1 <= 2: copia menor	[3, 4, 5, 1, 2]	4	0	6
14	Copia restante direito	[3, 4, 5, 1, 2]	4	0	7
15	Intercala [0,2] e [3,4]	[3, 4, 5, 1, 2]	4	0	7
16	3 <= 1: copia menor	[1, 4, 5, 1, 2]	5	0	8
17	3 <= 2: copia menor	[1, 2, 5, 1, 2]	6	0	9
18	Copia restante esquerdo	[1, 2, 3, 1, 2]	6	0	10
19	Copia restante esquerdo	[1, 2, 3, 4, 2]	6	0	11
20	Copia restante esquerdo	[1, 2, 3, 4, 5]	6	0	12
21	Resultado	[1, 2, 3, 4, 5]	6	0	12

Total: 6 comparações, 0 trocas e 12 movimentos.

Ordem das divisões: [5, 3, 4] e [1, 2]; depois [5, 3] e [4]; depois [5] e [3]. As intercalações produzem [3, 5], [3, 4, 5], [1, 2] e, por fim, o vetor inteiro ordenado.

Quick sort

Usei a partição de Lomuto, escolhendo o último elemento de cada trecho como pivô. Separei os valores menores ou iguais à esquerda, posicionei o pivô e repeti nos dois lados.

Entrada: [5, 3, 4, 1, 2] | Saída: [1, 2, 3, 4, 5]

Passo	Ação	Vetor	C	T	M
0	Entrada	[5, 3, 4, 1, 2]	0	0	0
1	Particao [0,4]; pivo=2	[5, 3, 4, 1, 2]	0	0	0
2	5 <= pivo 2: mantem	[5, 3, 4, 1, 2]	1	0	0
3	3 <= pivo 2: mantem	[5, 3, 4, 1, 2]	2	0	0
4	4 <= pivo 2: mantem	[5, 3, 4, 1, 2]	3	0	0
5	1 <= pivo 2: esquerda	[1, 3, 4, 5, 2]	4	1	0
6	Pivo no indice 1	[1, 2, 4, 5, 3]	4	2	0
7	Particao [2,4]; pivo=3	[1, 2, 4, 5, 3]	4	2	0
8	4 <= pivo 3: mantem	[1, 2, 4, 5, 3]	5	2	0
9	5 <= pivo 3: mantem	[1, 2, 4, 5, 3]	6	2	0
10	Pivo no indice 2	[1, 2, 3, 5, 4]	6	3	0
11	Particao [3,4]; pivo=4	[1, 2, 3, 5, 4]	6	3	0
12	5 <= pivo 4: mantem	[1, 2, 3, 5, 4]	7	3	0
13	Pivo no indice 3	[1, 2, 3, 4, 5]	7	4	0
14	Resultado	[1, 2, 3, 4, 5]	7	4	0

Total: 7 comparações, 4 trocas e 0 movimentos.